Predicting House Price with Trulia.com Data

Sam Fortescue, Mark Gardner, Nat Koonmee

12/16/21

Abstract

The price of any commodity is often only an approximation of its true underlying value. As a result, humans, when making purchasing decisions, should take caution to avoid paying too much. In the case of real estate commodities (i.e. houses), datasets from house listing websites are plentiful; for our analysis we used a dataset from Trulia.com (PromptCloud & DataStock, 2019), which contains around 30000 house listing in the United States from September-October 2019. Statistical learning offers a well defined framework for assisting in decisions, by comparing houses considered for purchase with others that are similar. Our goal was to predict house listing price using Multiple Linear Regression, LASSO and Ridge Regression, Principal Component and Partial Least Squares Regression, Generalized Additive Models (Polynomial Regression, Step Functions), and Tree Based Methods. We subset the dataset by city and state based on the assumption that housing markets tend to be local; predicting price for homes in California, Colorado, Los Angeles, and Denver. Models were trained on 70% of the data, and predictive performance was evaluated with Mean Squared Error calculated from the remaining 30%. There was a primary problem with the data that caused issues in analysis, namely there weren't enough observations of houses that were similar to eachother. In hindsight this issue is quite obvious since house listing websites only list houses for sale, and most houses are not for sale. A contrasting dataset would be the Ames dataset (De Cock, 2011), which contains details about all houses, whether for sale or not. Ultimately the test MSE values obtained were too large for our models to be considered useful in making house purchasing decisions.

Data Wrangling

Below is a brief summary of the datasets, post-cleaning, for help in understanding. Note the variables categorized by type:

Table 1: Data summary

Name	re
Number of rows	9999
Number of columns	19
Column type frequency:	

Table 1: Data summary

character	5
factor	2
numeric	12
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
Description	0	1	1	3000	0	9966	0
Address	0	1	23	57	0	9999	0
City	0	1	4	20	0	431	0
State	0	1	2	2	0	27	0
Zipcode	0	1	5	5	0	1717	0

Variable type: factor

skim_variable	n_missing	complete_rate	ordered	n_unique	top_counts
Beds	0	1	FALSE	15	3: 3992, 4: 2588, 2: 1603, 5: 1015
Bath	0	1	FALSE	30	2: 3634, 3: 2718, 1: 1556, 4: 1039

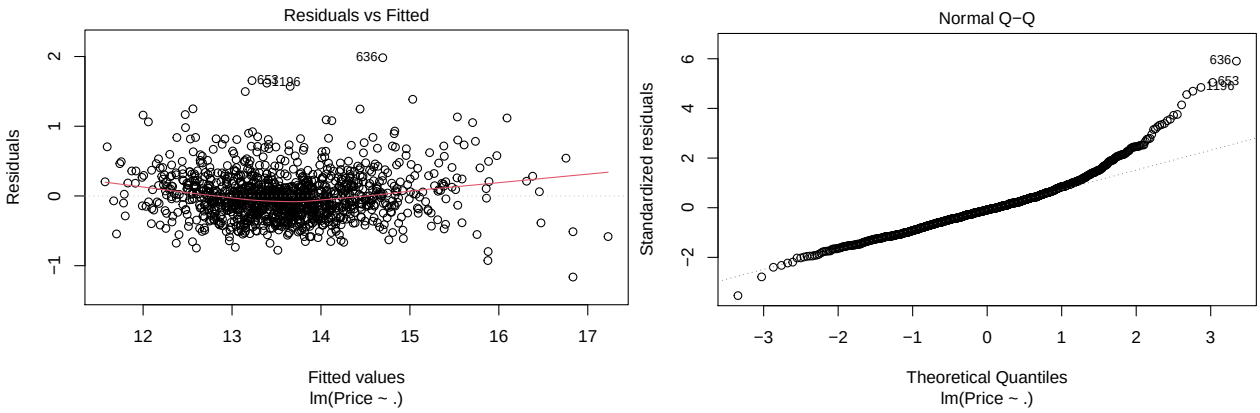
Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
Price	0	1	575445.01	137220.33	3000.00	209925.00	535000.00	580000.00	32500000.0	□□□□□
SqrFt	0	1	5766.36	360037.16	1.00	1284.00	1778.00	2600.00	36003799.0	□□□□□
Longitude	0	1	-95.22	16.50	-122.84	-112.14	-87.69	-81.33	-71.0	□□□□□
Latitude	0	1	36.97	4.92	25.55	33.76	36.26	40.58	47.8	□□□□□
LotSize	0	1	0.63	15.92	0.00	0.09	0.16	0.25	1260.0	□□□□□

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
YearBuilt	0	1	1968.51	33.96	1808.00	1946.00	1972.00	1999.00	2019.0	▁▁▅▅▅
PriceSqrFt	0	1	400.70	11495.62	0.00	126.00	175.00	275.00	1100000.0	▁▁▁▁▁
LastSoldYear	0	1	2011.06	6.67	1987.00	2006.00	2013.00	2017.00	2019.0	▁▁▅▅▅
LastSoldFor	0	1	361647.95	5350636.90	1.00	117000.00	215000.00	389850.00	1200000000.0	▁▁▁▁▁
LastTaxAssestment	0	1	256225.07	800965.82	0.00	28249.00	101370.00	258823.50	34252200.0	▁▁▁▁▁
LastTaxYear	0	1	2017.84	0.85	2010.00	2018.00	2018.00	2018.00	2019.0	▁▁▅▅▅
DaysOnTrulia	0	1	56.18	58.61	2.00	6.00	35.00	90.50	180.0	▁▁▅▅▅

The Trulia.com dataset (PromptCloud & DataStock, 2019) initially had 67 columns to consider as predictors, however many didn't have any predictive meaning i.e. `Unique House ID`. In addition, several predictors contained the same information as another, and several more were entirely composed of NA values. After the removal of these columns there were 18 potentially useful predictors. Two of these were `Address` and `Description`; we considered pulling additional features out of these e.g. an indicator if a description contained the word "beautiful," or street name. Given that there usually weren't multiple house listings on the same street, and beautiful is just word that anyone can use, we opted not to do these things. After pulling out useful predictors, we removed observations that had missing observations. In hindsight this may have been a mistake because as we will see in the methodology section below, we didn't end up using many predictors due to errors in analysis, so we could have potentially had more data. The remainder of the data cleaning consisted of removing unit labels, converting all `LotSize` observations to Acres as some were in SqrFt, getting rid of dollar signs, and making everything that should be a factor a factor (or character), and everything else numeric.

For linear models it was necessary to do several data transformations. Most notably we predicted log(`Price`) instead of `Price`, however we were still able to get interpretable predictions via inverse transformations. In general for predictor variables that represented a dollar amount, a log transform worked well. For `SqrFt` taking the square root worked well. We did not transform time data e.g. `LastTaxYear`, because doing log(`Price`) was enough, also it doesn't really make sense to do so. It should be noted that we did look at potentially doing different transformations for different subsets, it just turned out that the same transformations worked for each. The following diagnostic plots for a multiple linear regression on all numeric predictors from the California subset, show that the transformations worked reasonably well at making a linear model have lower bias:



To see our exact data wrangling process view the code appendix below under "# DATA WRANGLING".

Statistical Methodology

An interesting error arose when performing multiple linear regression in which dummy variables for different categorical predictors caused linear dependencies in the design matrix.

Results

The test-MSEs in log-price for each subset and method are given. In addition we provide the test-RMSE after performing the inverse log transformation i.e. e^x, on observed and predicted log-price.

Conclusions

References

De Cock, Dean. 2011. 'Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project.' Journal of Statistics Education, Volume 19, Number 3. Retrieved 12/14/2021 from (http://jse.amstat.org/v19n3/decock.pdf)

PromptCloud, & DataStock, 2019. Real Estate Data From Trulia (Version 1). A house listing dataset from Trulia.com. Retrieved 12/14/2021 from (https://www.kaggle.com/promptcloud/real-estate-data-from-trulia)

Code Appendix

```
# PACKAGES NEEDED
```

```r
library(readr)
library(ggplot2)
library(leaps)
library(glmnet)
library(car)
library(pls)
library(dplyr)
library(knitr)
library(skimr)
library(gridExtra)


# DATA WRANGLING CODE

# read in data
re <- read_csv("./realEstate.csv")

# get rid of useless columns
re <- re[,c(-(1:4),-7,-8,-12,-18,-(27:29),-(31:68))]

# remove observations with missing data
re <- na.omit(re)

# make better var names
names(re) <- gsub(" ","", names(re))

# remove unit labels and dollar signs
re$Price <- parse_number(re$Price)
re$SqrFt <- parse_number(re$SqrFt)

inSqft <- rep(NA,length(re$LotSize))
for (i in 1:length(re$LotSize)) {
  if (grepl("sqft",re$LotSize[i],fixed=TRUE)) {
    inSqft[i] <- TRUE
  } else {
    inSqft[i] <- FALSE
  }
}
re$LotSize <- parse_number(re$LotSize)
for (i in 1:length(re$LotSize)) {
  if (inSqft[i]) {
    re$LotSize[i] <- re$LotSize[i]*(1/43560)
  }
}

re$Bath <- as.factor(re$Bath)
re$Beds <- as.factor(re$Beds)

re$LastSoldFor <- parse_number(re$LastSoldFor)
re$LastTaxAssestment <- parse_number(re$LastTaxAssestment)
re$PriceSqrFt <- parse_number(re$PriceSqrFt)
```

```r
# SUBSETS

ca <- re[re$State == "CA",c(-1,-10,-15)]
co <- re[re$State == "CO",c(-1,-10,-15)]
la <- re[re$City == "Los Angeles",c(-1,-10,-15)]
den <- re[re$City == "Denver",c(-1,-10,-15)]


# DATA TRANSFORMATIONS

ca_t <- ca[,c(-7,-14,-15)]
ca_t$Price <- log(ca_t$Price)
ca_t$SqrFt <- sqrt(ca_t$SqrFt)
ca_t$LotSize <- log(ca_t$LotSize)
ca_t$LastSoldFor <- log(ca_t$LastSoldFor)
ca_t$LastTaxAssestment <- log(ca_t$LastTaxAssestment)
ca_t$LastTaxYear <- as.factor(ca_t$LastTaxYear)

co_t <- co[,c(-12,-14,-15)]
co_t$Price <- log(co_t$Price)
co_t$SqrFt <- sqrt(co_t$SqrFt)
co_t$LotSize <- log(co_t$LotSize)
co_t$LastSoldFor <- log(co_t$LastSoldFor)
co_t$LastTaxAssestment <- log(co_t$LastTaxAssestment)

la_t <- la[,c(-7,-13,-14,-15)]
la_t$Price <- log(la_t$Price)
la_t$SqrFt <- sqrt(la_t$SqrFt)
la_t$LotSize <- log(la_t$LotSize)
la_t$LastSoldFor <- log(la_t$LastSoldFor)
la_t$LastTaxAssestment <- log(la_t$LastTaxAssestment)
la_t$LastTaxYear <- as.factor(la_t$LastTaxYear)

den_t <- den[,c(-12,-13,-14,-15)]
den_t$Price <- log(den_t$Price)
den_t$SqrFt <- sqrt(den_t$SqrFt)
den_t$LotSize <- log(den_t$LotSize)
den_t$LastSoldFor <- log(den_t$LastSoldFor)
den_t$LastTaxAssestment <- log(den_t$LastTaxAssestment)


# DATA SUMMARY

skim(re)


# MLR California on Transformed Numeric Predictors

diag_lm_ca <- lm(Price~.,data=ca_t[,c(-6,-12)])


# DIAGNOSTIC PLOTS CALIFORNIA
```

```r
plot(diag_lm_ca,c(1))
plot(diag_lm_ca,c(2))
```